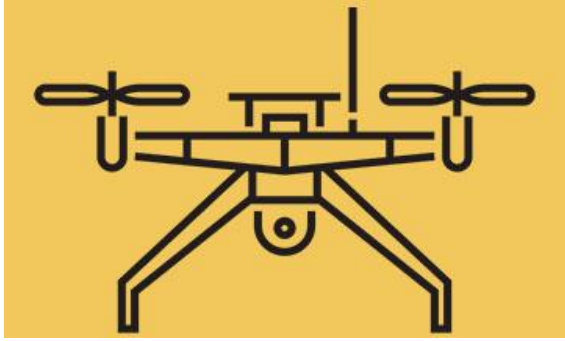
Kinetics

Jerome Jouffroy, Professor
jerome@sdu.dk

SDU

Gravity, propulsion and aerodynamics



So far, we have seen how to model and simulate:

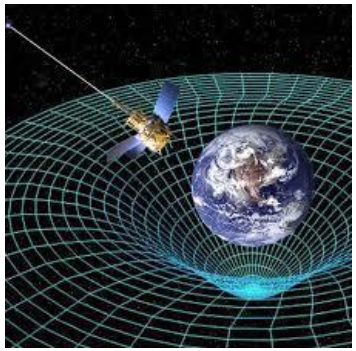
- The attitude and position of a UAV (kinematics)
- The dynamics expressed in the body frame (rigid-body kinetics)

Mathematical model:

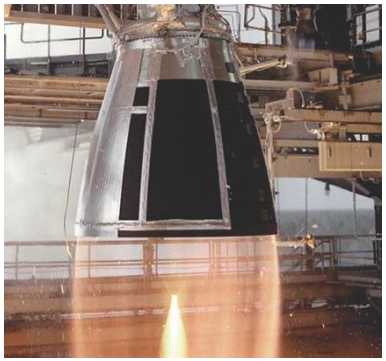
$$\begin{cases} \dot{\boldsymbol{\eta}} = \mathbf{J}_{\boldsymbol{\Theta}}(\boldsymbol{\eta})\boldsymbol{\nu} & \text{(kinematics)} \\ \mathbf{M}_{RB}\dot{\boldsymbol{\nu}} + \mathbf{C}_{RB}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau} & \text{(rigid-body kinetics)} \end{cases}$$

But to simulate a UAV more realistically, we need to add other forces and moments...

What is $\boldsymbol{\tau} = \begin{bmatrix} \mathbf{F}^b \\ \mathbf{m}^b \end{bmatrix}$?



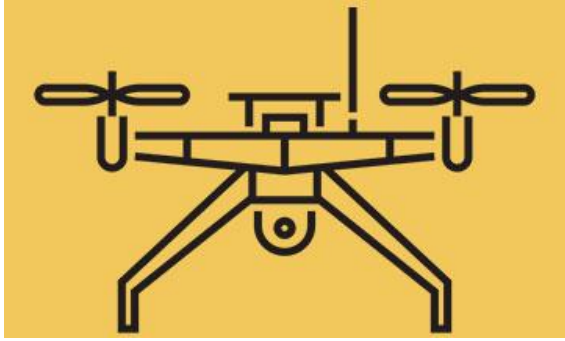
Gravity



Propulsion

Aerodynamics

Gravity



(we assume a constant gravity vector)

In the NED frame, we have

$$\mathbf{F}_g^e = \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix}$$

gravity

Hence, in the body-fixed frame, we have

$$\begin{aligned} \mathbf{F}_g^b &= \mathbf{R}_e^b \, \mathbf{F}_g^e \\ &= (\mathbf{R}_b^e)^T \, \mathbf{F}_g^e \end{aligned}$$

Thus, the translational rigid-body dynamics with gravity are

$$\boxed{m\dot{\mathbf{v}}_{b/e}^b + \boldsymbol{\omega}_{b/e}^b \times (m\mathbf{v}_{b/e}^b) = \mathbf{R}_e^b \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix}}$$

Remark: The gravity field does not induce a moment since we assume that the center of gravity of the body is the origin of the body-fixed frame.

Rotors

Main means of propulsion for UAVs

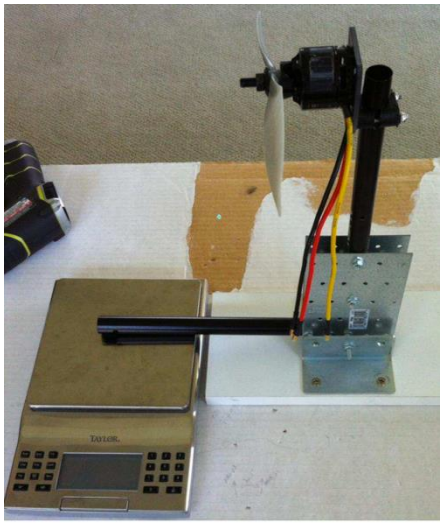
Rotors generate
- T: thrust perpendicular to propeller plane
- M: reaction moment due to propeller and motor shaft rotation

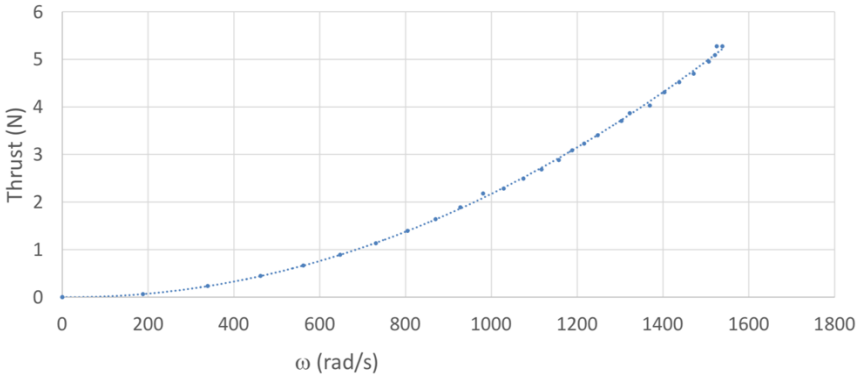
Simple mathematical model: $\boxed{M = k_M \Omega^2}$ and $\boxed{T = k_T \Omega^2}$

k_T, k_M constant, Ω angular velocity of propeller

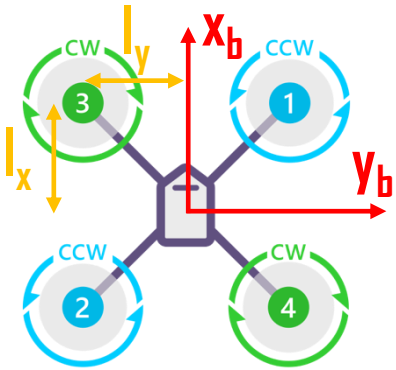
k_T and k_M and determined experimentally

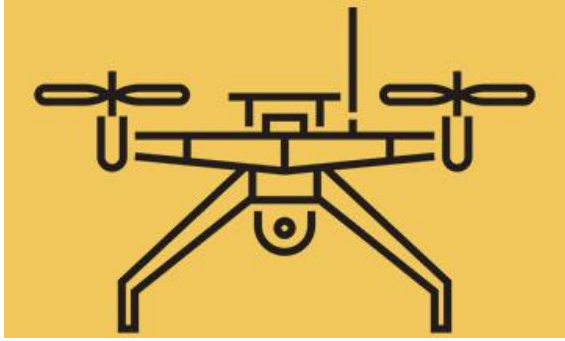
Ω is regulated using an ESC





Rotors on a quadcopter (1/2)





For F_z^b, we have $(F_x^b = F_y^b = 0)$

Quadcopter is <u>underactuated</u>!

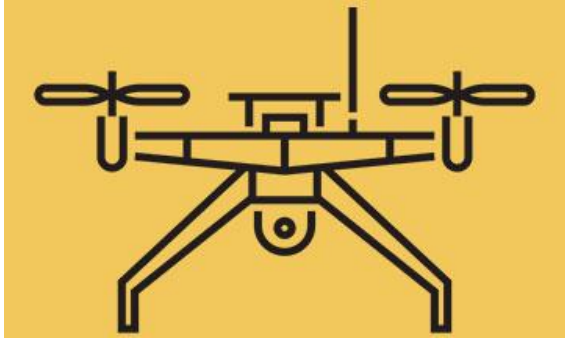
Can only move laterally by first pitching or rolling

$$F_z^b = -T_1 - T_2 - T_3 - T_4$$
$$= -k_T\Omega_1^2 - k_T\Omega_2^2 - k_T\Omega_3^2 - k_T\Omega_4^2$$

For the moments, we get

$$m_x^b = -l_yT_1 + l_yT_2 + l_yT_3 - l_yT_4$$
$$= -l_yk_T\Omega_1^2 + l_yk_T\Omega_2^2 + l_yk_T\Omega_3^2 - l_yk_T\Omega_4^2$$
$$m_y^b = l_xT_1 - l_xT_2 + l_xT_3 - l_xT_4$$
$$= l_xk_T\Omega_1^2 - l_xk_T\Omega_2^2 + l_xk_T\Omega_3^2 - l_xk_T\Omega_4^2$$
$$m_z^b = -M_1 - M_2 + M_3 + M_4$$
$$= -k_M\Omega_1^2 - k_M\Omega_2^2 + k_M\Omega_3^2 + k_M\Omega_4^2$$

Rotors on a quadcopter (2/2)



This gives

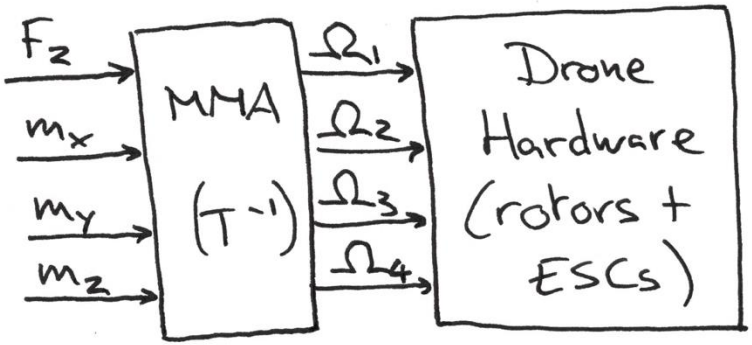
$$\underbrace{\begin{bmatrix} F_z^b \\ m_x^b \\ m_y^b \\ m_z^b \end{bmatrix}}_{\boldsymbol{\tau}_r} = \underbrace{\begin{bmatrix} -k_T & -k_T & -k_T & -k_T \\ -l_y k_T & l_y k_T & l_y k_T & -l_y k_T \\ l_x k_T & -l_x k_T & l_x k_T & -l_x k_T \\ -k_M & -k_M & k_M & k_M \end{bmatrix}}_{\mathbf{T}\ \text{Thrust matrix}} \underbrace{\begin{bmatrix} \Omega_1^2 \\ \Omega_2^2 \\ \Omega_3^2 \\ \Omega_4^2 \end{bmatrix}}_{\boldsymbol{\Omega}^2}$$

Since T is invertible, one can control F_z, M_x, M_y, M_z directly:

$$\boldsymbol{\tau}_r = \mathbf{T}\,\boldsymbol{\Omega}^2$$

$$\boldsymbol{\Omega}^2 = \mathbf{T}^{-1}\,\boldsymbol{\tau}_r$$

<u>Motor Mixing Algorithm</u>: how to link motor velocities to forces and moments applied to the CoG of the drone



What if we have more propellers? (eg. Hexacopter)

$$\mathbf{T} \in \mathbb{R}^{4\times 6}$$

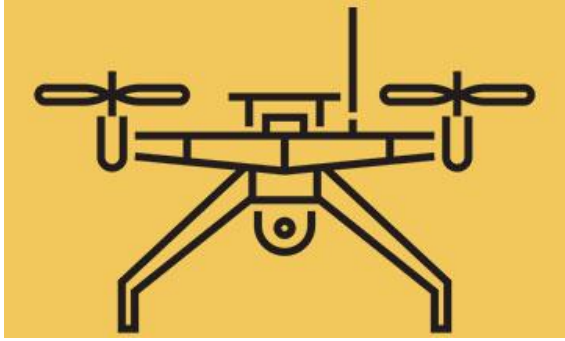
Pseudo-inverse

$$\mathbf{T}^{+} = \mathbf{T}^{T}\left(\mathbf{T}\mathbf{T}^{T}\right)^{-1}$$

Remark: Algorithms for MMA can be more advanced and take into account other aspects (saturation, fault on one rotor, etc.).
=> thrust allocation (constrained optimization)

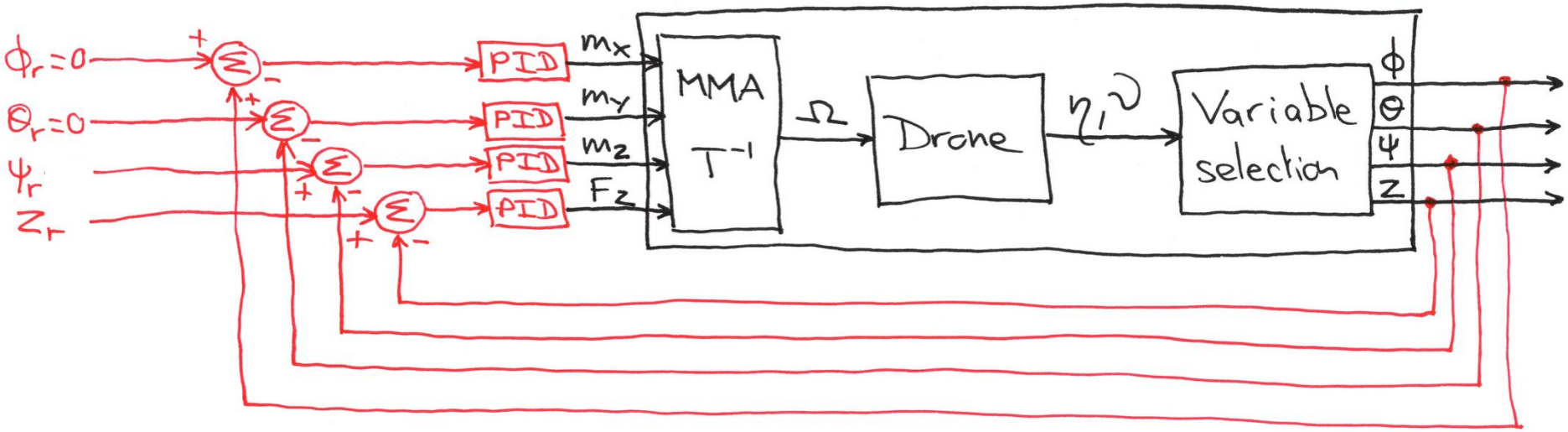
SDU

PID control of a quadcopter (1/3)

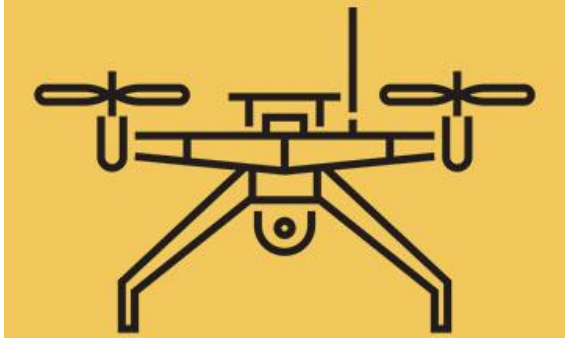


How can we use the previous results to stabilize a quadcopter (in a simple way)?

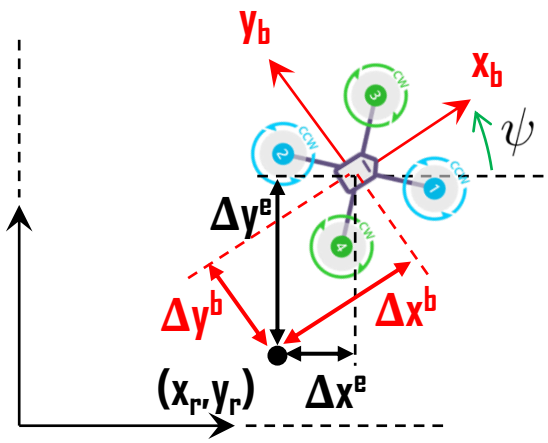
Idea: assign a PID controller to stabilize each DOF separately



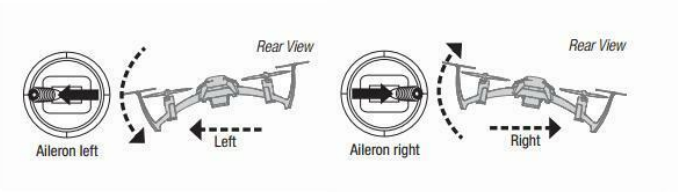
PID control of a quadcopter (2/3)



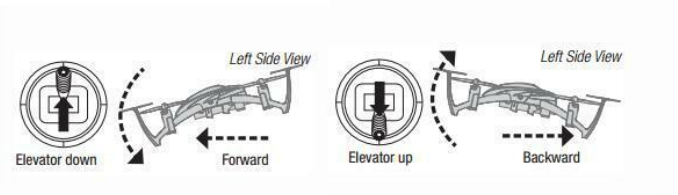
What about x,y stabilization?



Non-zero pitch angle makes drone go forward or backward



Non-zero roll angle angle makes drone go laterally left or right



Let (x_r, y_r) be a reference point the drone should reach

Let
$$\Delta x^e := x_r - x^e$$
$$\Delta y^e := y_r - y^e$$
Expressed in the {e} frame

Problem: all of the above is not done in the earth-fixed frame...

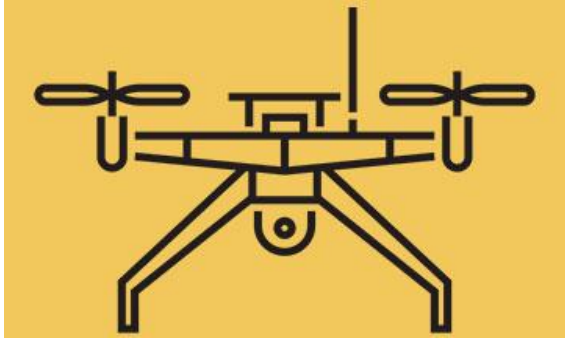
in the {b} frame:
$$\begin{bmatrix} \Delta x^b \\ \Delta y^b \end{bmatrix} = \begin{bmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} \Delta x^e \\ \Delta y^e \end{bmatrix}$$

OR
$$\begin{bmatrix} \Delta x^b \\ \Delta y^b \end{bmatrix} = \mathbf{R}^T(\psi) \begin{bmatrix} \Delta x^e \\ \Delta y^e \end{bmatrix}$$

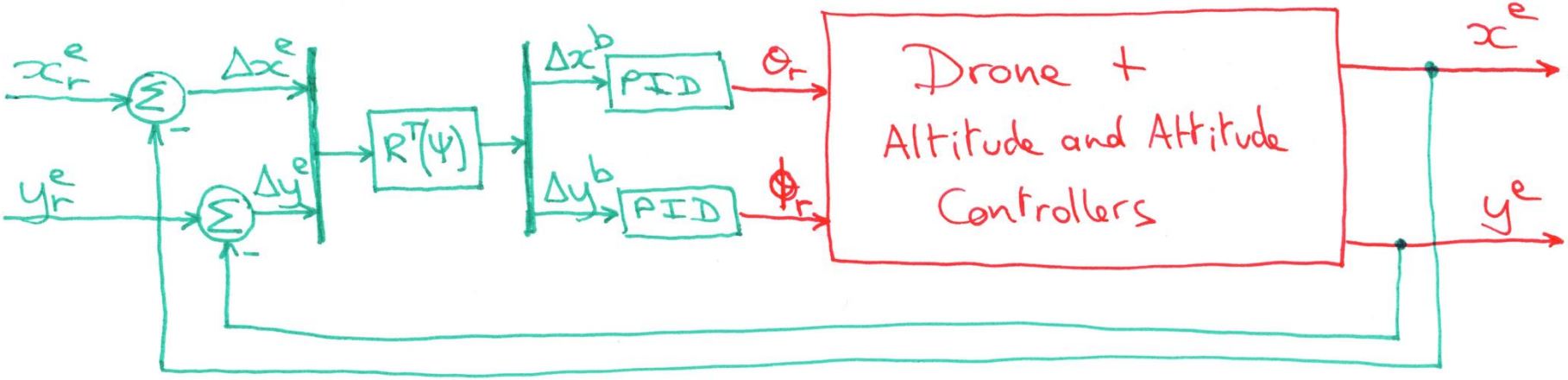
AND changing ϕ_r, θ_r changes $\Delta x^b, \Delta y^b$

SDU

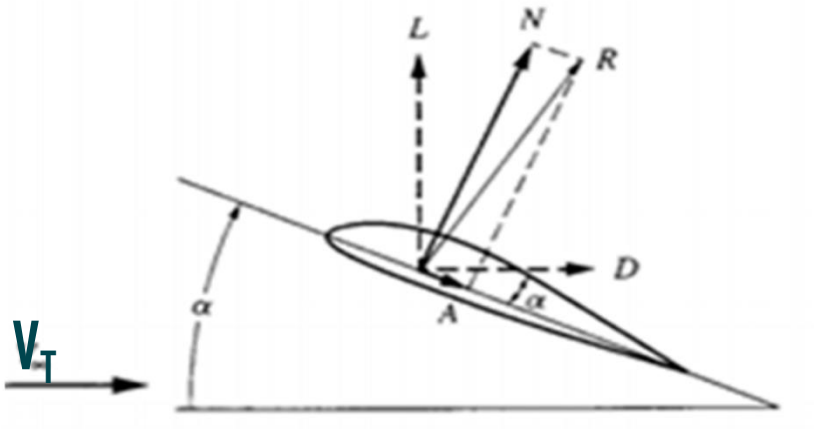
PID control of a quadcopter (3/3)

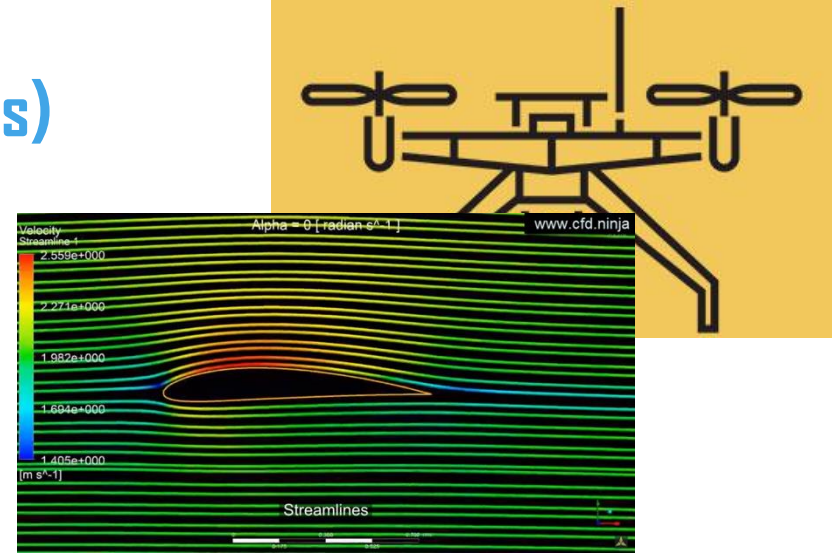


Hence we have the outer-loop position controller:



Aerodynamics in 2D: lift and drag (forces)





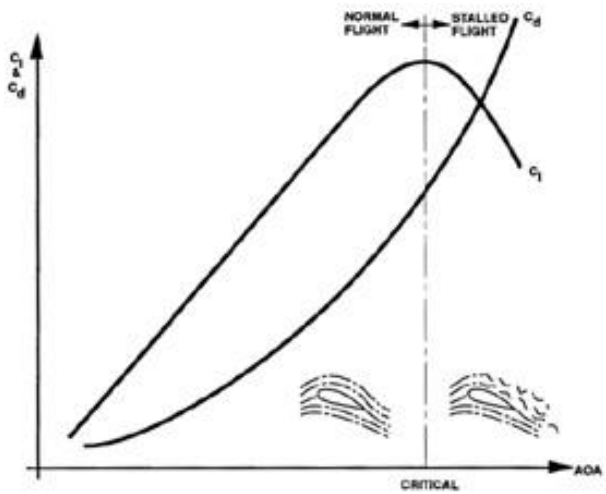
The aerodynamic force applied to the aerodynamic center (AC) can be decomposed into

Lift (force) $$L = C_L(\alpha)\, Q\, A$$

Wing area

with $Q = \frac{1}{2}\rho V_T^2$ dynamic pressure

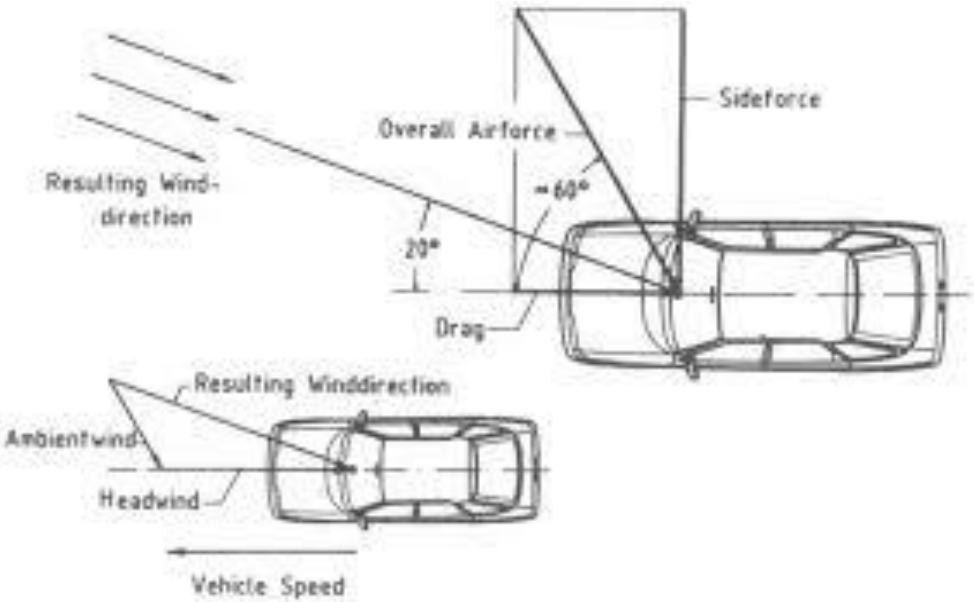
Drag (force) $$D = C_D(\alpha)\, Q\, A$$

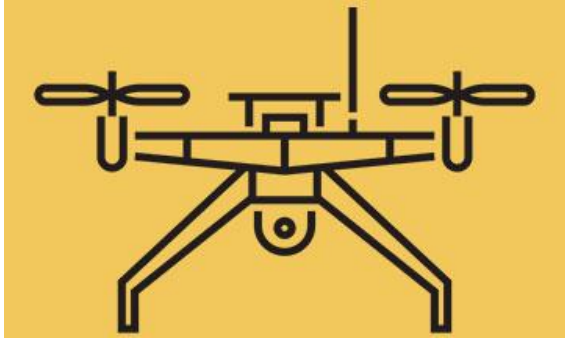


SDU

Both depend on the angle of attack (AOA) α

Aerodynamics in 2D: drag and side forces





Similarly, for a ground vehicle evolving on the horizontal plane, the aerodynamic forces will depend on the orientation (sideslip angle β) with respect to the flow

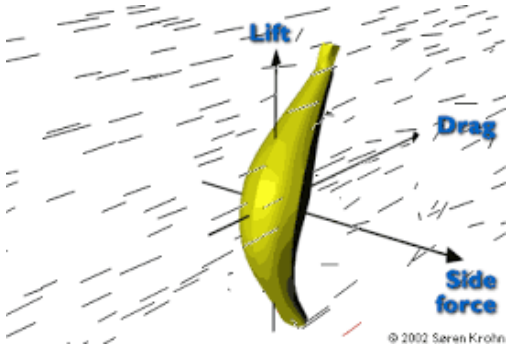
Drag (force)

$$D = C_D(\beta)\, Q\, A$$

Side force

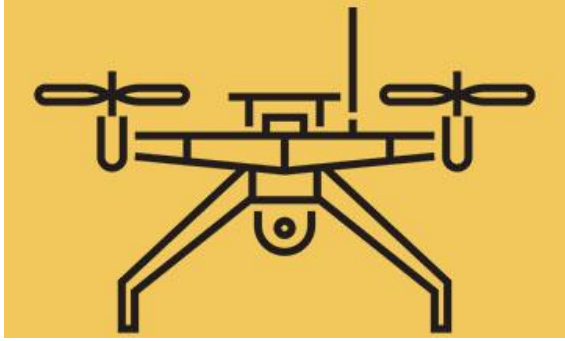
$$Y = C_Y(\beta)\, Q\, A$$

Hence, in 3D, we will have the 3 aerodynamic forces

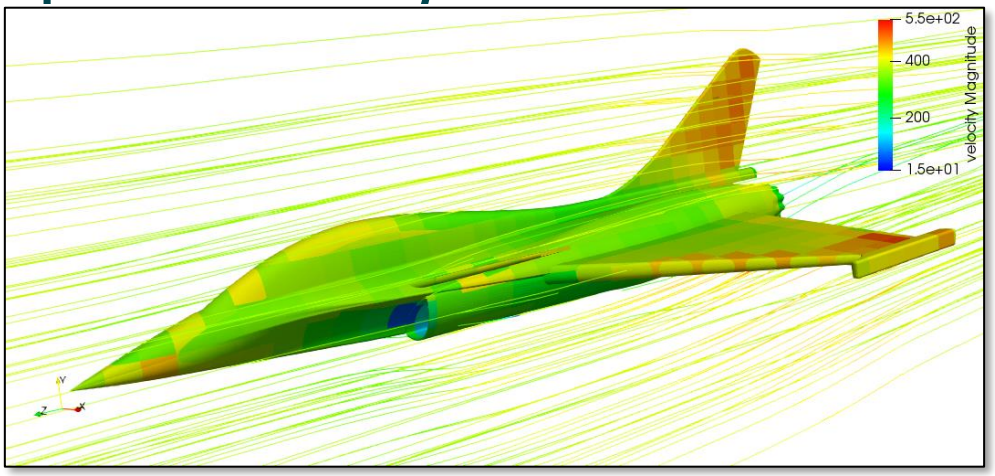


SDU

How are the lift, drag, side forces obtained?



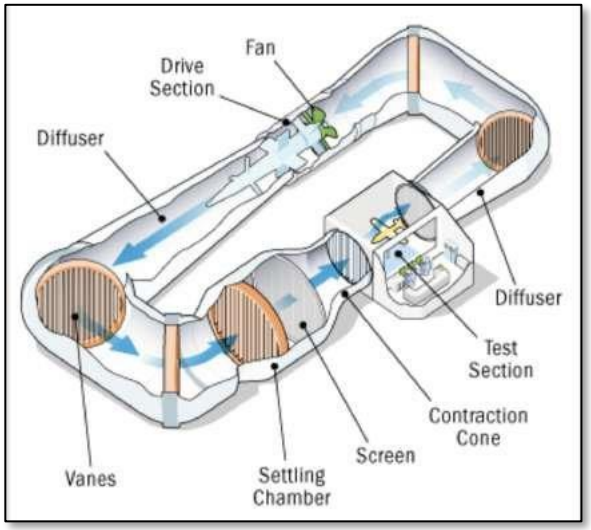
Computational Fluid Dynamics



ANSYS® FLUENT®

flow5

Wind Tunnel Experiments



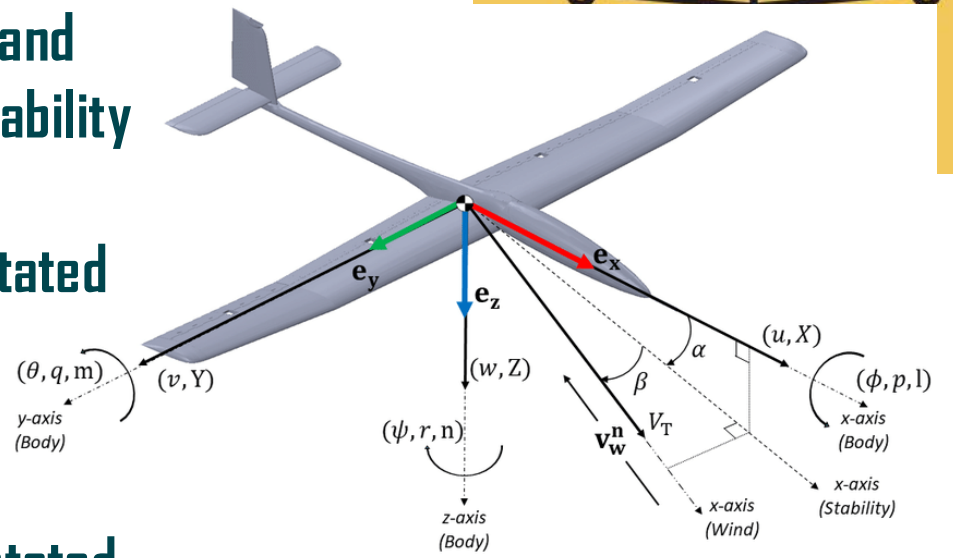
SDU

The 3D case: Stability and Wind axes

Similarly to the 2D case, the lift, drag and side forces will be expressed in the stability axis (or the wind axis)

The stability axis is the body frame rotated by the angle of attack α, with

$$\tan\alpha = \frac{w}{u}$$



The wind axis is the stability frame rotated by the sideslip angle β, with

$$\sin\beta = \frac{v}{V_T}$$

with the airspeed

$$V_T = \sqrt{u^2 + v^2 + w^2}$$

The aerodynamic force F_a, expressed in the stability frame, is given by

$$\mathbf{F}_a^s = \begin{bmatrix} -D \\ Y \\ -L \end{bmatrix} = \begin{bmatrix} -C_D\, Q\, A \\ C_Y\, Q\, A \\ -C_L\, Q\, A \end{bmatrix}$$

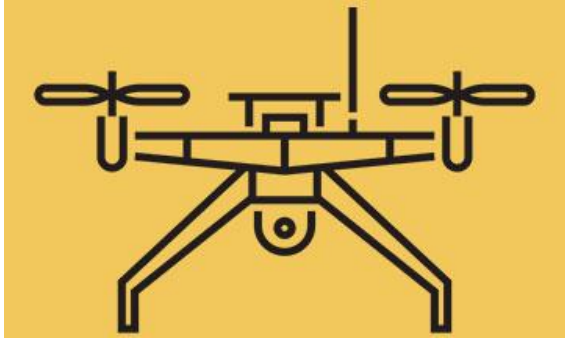
which gives, back in the {b} frame:

$$\mathbf{F}_a^b = \mathbf{R}_s^b\, \mathbf{F}_a^s \quad \text{with} \quad \mathbf{R}_s^b(\alpha) = \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha \\ 0 & 1 & 0 \\ \sin\alpha & 0 & \cos\alpha \end{bmatrix}$$

Remark: C_L, C_D, C_Y typically depend on many variables, including aileron angle, rudder angle, V_T, AOA, etc.

SDU

Putting everything together



Including kinematics, rigid-body kinetics, gravity, propulsion and aerodynamics, a UAV in flight can then be modelled by

$$\boxed{\begin{cases} \dot{\boldsymbol{\eta}} = \mathbf{J}_{\boldsymbol{\Theta}}(\boldsymbol{\eta})\boldsymbol{\nu} \\ \mathbf{M}_{RB}\dot{\boldsymbol{\nu}} + \mathbf{C}_{RB}(\boldsymbol{\nu})\boldsymbol{\nu} = \boldsymbol{\tau}_g + \boldsymbol{\tau}_p + \boldsymbol{\tau}_a \end{cases}}$$

(external disturbances are not represented)

with

$$\boldsymbol{\tau}_g = \begin{bmatrix} \mathbf{F}_g^b \\ 0 \end{bmatrix}$$

Gravity

$$\boldsymbol{\tau}_p = \begin{bmatrix} \mathbf{F}_p^b \\ \mathbf{m}_p^b \end{bmatrix}$$

Propulsion (rotors, but also jet engines and rocket engines)

$$\boldsymbol{\tau}_a = \begin{bmatrix} \mathbf{F}_a^b \\ \mathbf{m}_a^b \end{bmatrix}$$

Aerodynamics (wings, steering surfaces)

Examples of quadcopter models from the literature

A. Rigid-Body Model

A. Mathematical Model

As shown in Fig. 1, several
ered to describe the pose of a qu
E_I, the platform frame E_P, and
rotation of E_B relative to E_I i

For a quadrotor UAV, its
dynamics are described as the

$$\begin{cases} \dot{\boldsymbol{r}} = \boldsymbol{v} \\ m\dot{\boldsymbol{v}} = \boldsymbol{F}_c + \end{cases}$$

$$\begin{cases} \dot{\boldsymbol{q}} = \boldsymbol{q} \otimes [0 \\ \boldsymbol{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega}^{\times}\boldsymbol{J}\boldsymbol{\omega} \end{cases}$$

where $\boldsymbol{F}_c = \boldsymbol{R}_B^I \boldsymbol{T} + \boldsymbol{G}$, $\boldsymbol{G}$ is t
and $\boldsymbol{\omega}^{\times}$ is the skew-symmetric
$\boldsymbol{F}_d$ and $\boldsymbol{\tau}_d$ include aerodynam

II. Modeling

Consider a quadrotor vehicle formed by a rigid cross-shaped frame equipped with four rotors, as shown in Fig. 1. The system can be modeled as a rigid body of mass m and tensor of inertia I, subject to external forces and moments [25]–[28].

To describe the motion of the quadrotor, two reference frames are introduced: the inertial reference frame $\{\mathcal{I}\}$ fixed to the earth surface and associated with the vector basis $[e_1, e_2, e_3]$, and the body-fixed frame $\{\mathcal{B}\}$ attached to the quadrotor's center of gravity and associated with the vector basis $[{}^{\mathcal{B}}e_1, {}^{\mathcal{B}}e_2, {}^{\mathcal{B}}e_3]$. Let $(\xi, R) \in \mathbb{R}^3 \times SO(3)$ denote the configuration of $\{\mathcal{B}\}$ with respect to $\{\mathcal{I}\}$. Let V and Ω denote the translational and angular velocities, respectively, both expressed in $\{\mathcal{B}\}$. Then, the kinematics and dynamics of a quadrotor vehicle can be written as

$$\dot{\xi} = RV \tag{1}$$

$$m\dot{V} = -m\Omega_{\times}V + F + R^T\mu_w \tag{2}$$

$$\dot{R} = R\Omega_{\times} \tag{3}$$

$$I\dot{\Omega} = -\Omega_{\times}I\Omega + \Gamma \tag{4}$$

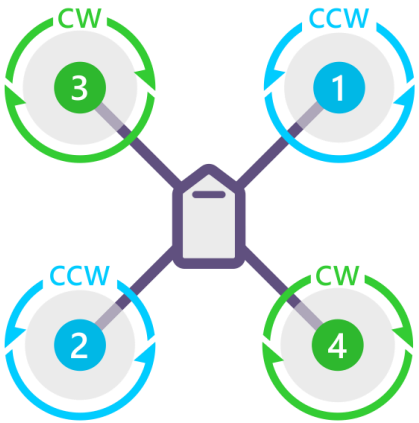
where F and Γ, both expressed in $\{\mathcal{B}\}$, are the force and torque applied to the vehicle, respectively. The matrix $\Omega_{\times}$ is a skew-symmetric matrix that satisfies $\Omega_{\times}b = \Omega \times b$, and the term μ_w, expressed in $\{\mathcal{I}\}$, accounts for disturbances, for example due to wind, and unmodeled dynamics.

The force combines thrust and gravity components and can be described as

$$F = -T^{\mathcal{B}}e_3 + mgR^T e_3 \tag{5}$$

Let us take a break

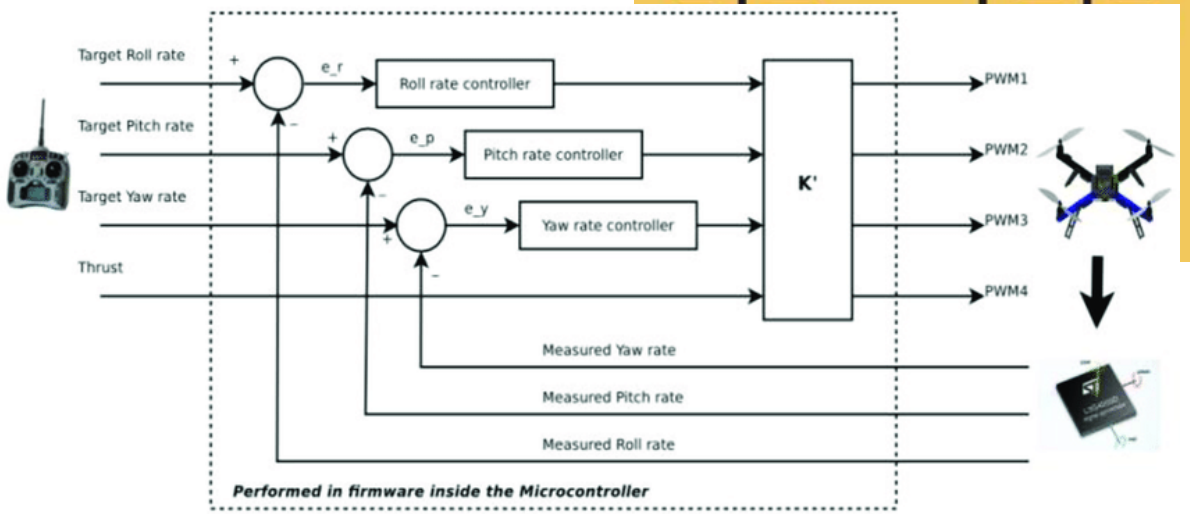
Today's exercise (1/3)



QUAD X

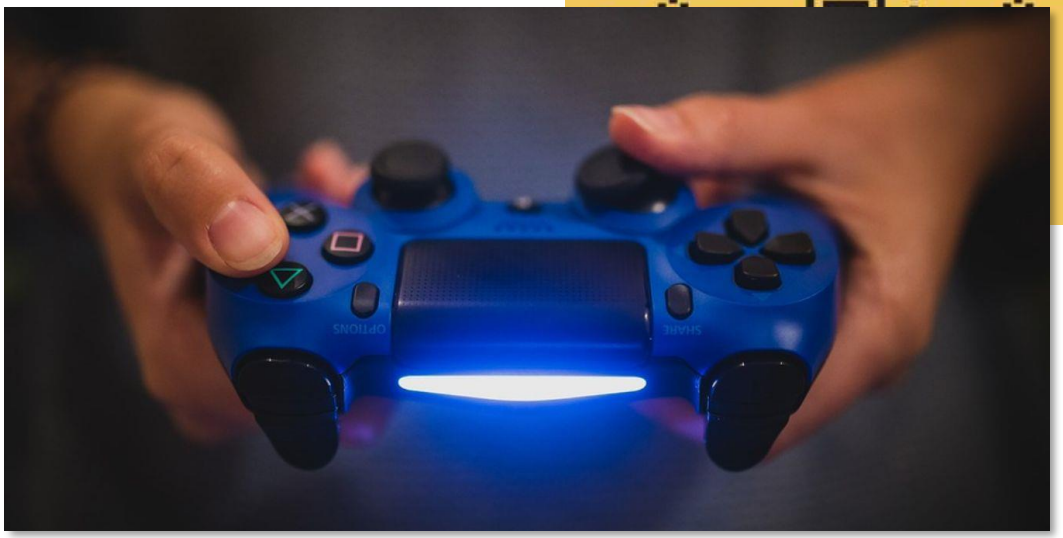
- Q1) From Itslearning, download the zip file quadcopter, decompress it in your currently active Matlab folder. Open the file quadcopter_temp.slx and check that it works.
- Q2) Implement a mathematical model of a quadrotor drone (<u>without gravity or thrust matrix</u>) using simple physical parameters, such as those of Q1) from Lecture 2 (note: do no re-invent the wheel, ie feel free to re-use previous programs/solutions to exercises).

Today's exercise (2/3)



- Q3) implement 4 PID inner-loop controllers. Find a way to change initial conditions so that the controllers are tuned separately and in this order: altitude, yaw, roll, pitch.
- Q4) add gravity to your model. add a pre-compensation algorithm into your controller to oppose gravity.
- Q5) We would like to stabilize the horizontal position of the drone. Add 2 outer-loop PID controllers so that the drone is regulated around reference positions x_r, y_r (put saturations after the controllers to avoid too much pitch or roll).

Today's exercise (3/3)



- Q6) We would like to gradually replace pre-defined reference signals with real-time inputs from a gamepad or sliders. Modify first your FCS so that you can control the altitude of your drone with a slider.
- Q7) Add a gamepad/slider feature for yaw.
- Q8) Implement a scheme so that you can control both the drone longitudinal and lateral velocities.
- Q9) Record a video of the drone moving under your control.